

Autenticación HTTP Basic

¿Qué es?

Es un mecanismo de autenticación integrado directamente en el protocolo HTTP, donde el cliente envía las credenciales del usuario (nombre de usuario y contraseña) codificadas en Base64 dentro del encabezado Authorization (Basic <credenciales>) en cada petición.

Problema que resuelve

Permite proteger endpoints de forma rápida y sencilla sin necesidad de implementar complejos sistemas de inicio de sesión, bases de datos de tokens o manejo de sesiones en el servidor.

Qué pasa si no se usa

Cualquier persona en internet podría invocar libremente los endpoints de tu API o aplicación, exponiendo información confidencial o permitiendo la ejecución de operaciones no autorizadas sin verificar la identidad del cliente.

```
14 @Configuration
15 public class SecurityConfig {
16
17     @Bean
18     UserDetailsService userDetailsService(DataSource theDataSource) {
19         JdbcUserDetailsManager theUserDetailsManager = new JdbcUserDetailsManager(theDataSource);
20         theUserDetailsManager.setUsersByUsernameQuery(
21             "select user_id, pw, active from members where user_id=?");
22         theUserDetailsManager.setAuthoritiesByUsernameQuery(
23             "select user_id, role from roles where user_id=?");
24         return theUserDetailsManager;
25     }
26
27     @Bean
28     SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
29         http.authorizeHttpRequests(configurer -> configurer
30             .requestMatchers(HttpMethod.GET, "/api/employees").hasRole("EMPLOYEE")
31             .requestMatchers(HttpMethod.GET, "/api/employees/**").hasRole("EMPLOYEE")
32             .requestMatchers(HttpMethod.POST, "/api/employees").hasRole("MANAGER")
33             .requestMatchers(HttpMethod.PUT, "/api/employees").hasRole("MANAGER")
34             .requestMatchers(HttpMethod.PATCH, "/api/employees/**").hasRole("MANAGER")
35             .requestMatchers(HttpMethod.DELETE, "/api/employees/**").hasRole("ADMIN")
36             .anyRequest().authenticated());
37         http.httpBasic(Customizer.withDefaults());
38         http.csrf(csrf -> csrf.disable());
39         http.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
40         return http.build();
41     }
42 }
43
```

Este código configura la seguridad de una API REST en Spring Boot utilizando Spring Security 6+. En palabras sencillas: conecta la seguridad a tu base de datos y define quién puede hacer qué con los empleados.

Las anotaciones @Configuration y @Bean trabajan en equipo para decirle a Spring cómo debe crear, configurar y guardar objetos clave en su memoria central (conocida como el Contexto de Spring).

@Configuration (La fábrica): Se coloca encima de una clase. Le avisa a Spring: "Esta clase no hace tareas del día a día; es un manual de instrucciones. Léela al arrancar la aplicación porque aquí te explico cómo armar piezas importantes del sistema".

@Bean (El producto fabricado): Se coloca encima de un método dentro de esa clase. Le dice a Spring: "Ejecuta este método, toma el objeto que devuelve y guárdalo en tu caja de herramientas. De ahora en adelante, si cualquier otra parte del sistema necesita este objeto, pásale este mismo en lugar de crear uno nuevo".

Cómo se conectan en el código de seguridad:

Cuando pusiste `@Configuration` en tu clase `SecurityConfig`, Spring supo que debía analizarla al encender el servidor. Al leerla, encontró dos métodos marcados con `@Bean`:

- El método que devuelve el `UserDetailsService` (que le enseña a buscar usuarios en la base de datos).
- El método que devuelve el `SecurityFilterChain` (que tiene las reglas de qué rutas están bloqueadas o permitidas).

Spring ejecutó ambos métodos de inmediato, tomó esos dos "objetos" resultantes y los conectó internamente al motor de Spring Security. Gracias a eso, tu aplicación sabe automáticamente cómo defenderse sin que tú tengas que llamar a esos métodos manualmente.

El código se divide en dos Beans principales:

Conexión a la Base de Datos (`UserDetailsService`)

Aquí se le está diciendo a Spring Security: "No busques usuarios en memoria, búscalos en la base de datos usando SQL personalizado".

`JdbcUserDetailsManager`: Le enseña a Spring a leer usuarios desde una base de datos relacional.

`setUsersByUsernameQuery(...)`: Es la consulta SQL para autenticar al usuario. Spring espera exactamente 3 campos en ese orden: usuario, contraseña y estado activo (boolean). En este caso, los busca en la tabla `members`.

`setAuthoritiesByUsernameQuery(...)`: Es la consulta SQL para obtener los permisos (roles) del usuario desde la tabla `roles`.

Reglas de Acceso y Reglas HTTP (`SecurityFilterChain`)

Aquí se define el control de acceso a los endpoints y el comportamiento de la sesión:

A) Control de Permisos por Roles:

GET: Solo lectura. Requiere el rol `ROLE_EMPLOYEE` (Spring agrega automáticamente el prefijo `ROLE_` automáticamente).

POST / PUT / PATCH: Crear y modificar empleados. Requiere el rol ROLE_MANAGER.

DELETE Eliminar empleados. Operación destructiva, requiere el rol ROLE_ADMIN.

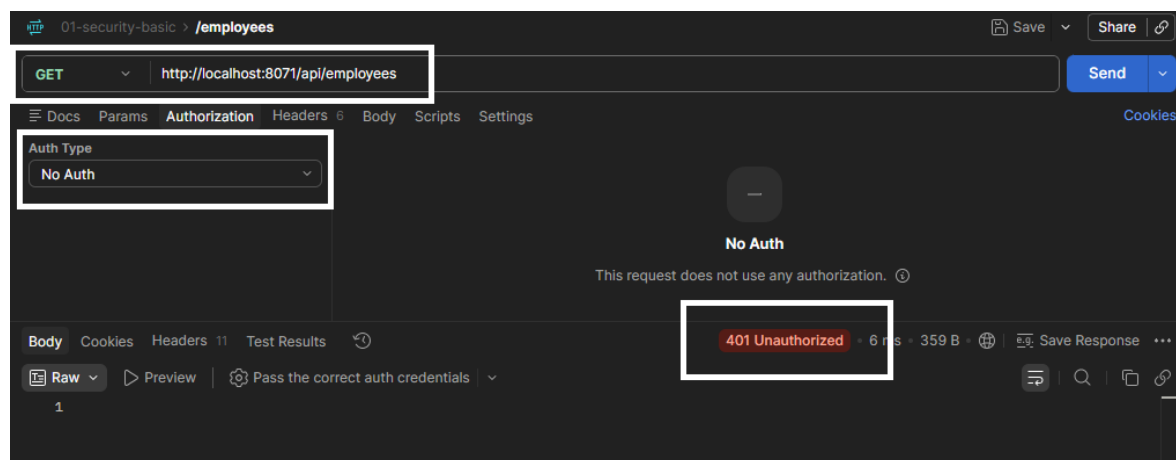
anyRequest().authenticated(): Cualquier otra ruta no especificada requerirá que el usuario esté logueado.

B) Mecánica de Seguridad:

httpBasic(...): Utiliza la autenticación HTTP Basic (el cliente envía usuario y contraseña en las cabeceras de cada petición).

csrf.disable(): Desactiva la protección CSRF. Es la práctica estándar para APIs REST stateless donde no utilizas sesiones de navegador basadas en Cookies.

SessionCreationPolicy.STATELESS: Le ordena a Spring Security nunca crear ni guardar una sesión HTTP en el servidor. Cada petición se trata de manera independiente y debe incluir sus credenciales.



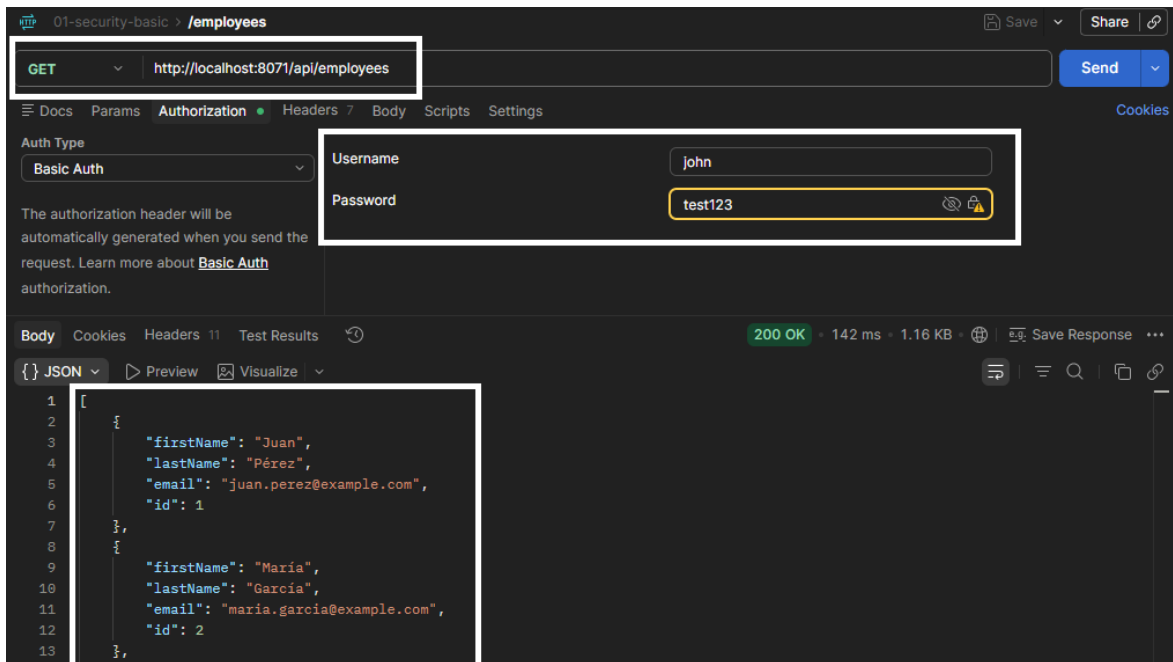
La imagen muestra cómo actúa el filtro de Spring Security cuando se intenta acceder a un recurso protegido sin usar HTTP Basic:

Petición (Recuadro superior): Se está ejecutando un método GET hacia la ruta `http://localhost:8071/api/employees`. Según la configuración de tu código anterior, esta ruta exige que el usuario esté autenticado y posea el rol EMPLOYEE.

Ausencia de credenciales (Recuadro izquierdo): En la pestaña de Autorización, el tipo está configurado como "No Auth". Esto indica que el cliente no está generando ni enviando la cabecera `Authorization: Basic` con el usuario y la contraseña.

Bloqueo por el servidor (Recuadro inferior derecho): Al recibir la petición sin credenciales, el `SecurityFilterChain` de Spring Boot la intercepta y la rechaza inmediatamente devolviendo un estado

401 Unauthorized (No autorizado). La aplicación ni siquiera llega a ejecutar la lógica para buscar empleados.



La imagen muestra una solicitud exitosa utilizando autenticación HTTP Basic en Spring Security:

Petición (Recuadro superior): Se mantiene la misma solicitud GET a la ruta `http://localhost:8071/api/employees`.

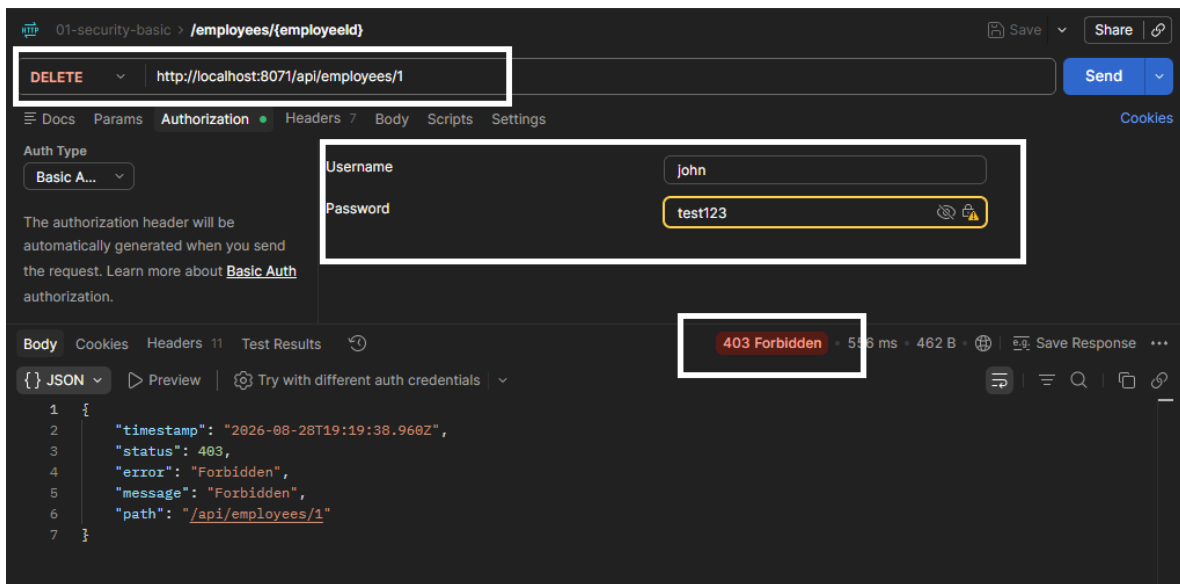
Credenciales configuradas (Recuadro medio): En la pestaña de Autorización, el tipo de autenticación cambió a Basic Auth. Se envían las credenciales del usuario: Username: john y Password: test123.

Autenticación y Autorización exitosas:

- Spring Security captura la cabecera Authorization Basic que Postman genera automáticamente a partir de ese usuario y contraseña.
- Valida contra la base de datos que john existe y que la contraseña coincide.
- Comprueba que el usuario john tiene asignado el rol necesario (ROLE_EMPLOYEE) para consultar esa ruta.

Respuesta exitosa (Recuadro inferior y estado):

- El servidor devuelve un código de estado 200 OK.
- En el cuerpo de la respuesta (Body), se entrega la lista de empleados en formato JSON (con los registros de Juan Pérez y María García).



La imagen ilustra un escenario de Autenticación exitosa pero Autorización denegada (Error 403) en Spring Security:

Petición (Recuadro superior): El cliente intenta ejecutar un método DELETE hacia la ruta `http://localhost:8071/api/employees/1` con la intención de borrar un registro.

Credenciales (Recuadro medio): Se están utilizando las mismas credenciales del ejemplo anterior mediante Basic Auth (Username: john, Password: test123).

Bloqueo por falta de permisos (Recuadro inferior): El servidor responde con un código 403 Forbidden (Prohibido) y un JSON detallando el error.

- A diferencia de la primera imagen (donde hubo un 401), aquí Spring Security sí reconoció al usuario y validó su contraseña correctamente.
- El bloqueo ocurre en la etapa de autorización. Según las reglas del `SecurityFilterChain` que revisamos antes, las peticiones DELETE exigen tener el rol ADMIN (`.hasRole("ADMIN")`). Como john probablemente solo tiene el rol EMPLOYEE, el sistema le prohíbe realizar esta acción destructiva.

Autenticación JWT (JSON Web Token)

¿Qué es?

Es un estándar abierto (RFC 7519) que define un formato compacto y seguro para transmitir información entre partes como un objeto JSON. Los datos están firmados digitalmente (lo que garantiza su integridad) y contienen las declaraciones o claims del usuario (como su ID, roles y tiempo de expiración).

Problema que resuelve

Resuelve el problema de la escalabilidad y el almacenamiento de sesiones en arquitecturas distribuidas (como microondas o APIs REST con frontends desacoplados). Con sesiones tradicionales, el servidor debe guardar el estado de la sesión en memoria o en una base de datos compartida (como Redis). Con JWT, el servidor valida la firma criptográfica del token recibido sin necesidad de consultar ninguna base de datos, logrando que la autenticación sea completamente libre de estado (stateless).

Qué pasa si no se usa

Si construyes una API REST moderna o una arquitectura de microservicios y no usas tokens como JWT, te verías obligado a depender de cookies de sesión tradicionales. Esto dificulta el balanceo de carga y el escalado horizontal, ya que cada instancia del servidor necesitaría acceder al repositorio centralizado de sesiones para reconocer al usuario.

```
33 @Configuration
34 public class SecurityConfig {
35
36     @Value("${rsa.public-key}")
37     private RSAPublicKey publicKey;
38
39     @Value("${rsa.private-key}")
40     private RSAPrivateKey privateKey;
41
42     @Bean
43     public UserDetailsService userDetailsService(DataSource theDataSource) {
44
45         JdbcUserDetailsManager theUserDetailsManager = new JdbcUserDetailsManager(theDataSource);
46
47         theUserDetailsManager.setUsersByUsernameQuery(
48             "select user_id, pw, active from members where user_id=?");
49
50         theUserDetailsManager.setAuthoritiesByUsernameQuery(
51             "select user_id, role from roles where user_id=?");
52
53         return theUserDetailsManager;
54     }
55 }
```

userDetailsService (DataSource theDataSource): El método configura un administrador de usuarios basado en bases de datos relacionales (JdbcUserDetailsManager), estableciendo las consultas SQL exactas que Spring Security debe ejecutar contra la base de datos inyectada para validar la contraseña y el estado del usuario en la tabla members, así como para recuperar sus permisos asociados desde la tabla roles.

```

56 @Bean
57 @Order(1)
58 public SecurityFilterChain loginFilterChain(HttpSecurity http) throws Exception {
59
60     http.securityMatcher("/api/auth/**");
61     http.authorizeHttpRequests(configurer -> configurer.anyRequest().authenticated());
62     http.httpBasic(Customizer.withDefaults());
63     http.csrf(csrf -> csrf.disable());
64     http.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
65
66     return http.build();
67 }
68

```

loginFilterChain (HttpSecurity http): El método define la primera barrera de seguridad del sistema mediante un filtro de alta prioridad (@Order(1)) dedicado exclusivamente a interceptar las rutas de inicio de sesión (/api/auth/**), donde se exige que el cliente envíe sus credenciales mediante autenticación HTTP Basic (para posteriormente generarle un token), operando de forma completamente stateless y sin protección CSRF.

```

69 @Bean
70 @Order(2)
71 public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
72
73     http.authorizeHttpRequests(configurer -> configurer
74         .requestMatchers(HttpMethod.GET, "/api/employees").hasRole("EMPLOYEE")
75         .requestMatchers(HttpMethod.GET, "/api/employees/**").hasRole("EMPLOYEE")
76         .requestMatchers(HttpMethod.POST, "/api/employees").hasRole("MANAGER")
77         .requestMatchers(HttpMethod.PUT, "/api/employees").hasRole("MANAGER")
78         .requestMatchers(HttpMethod.PATCH, "/api/employees/**").hasRole("MANAGER")
79         .requestMatchers(HttpMethod.DELETE, "/api/employees/**").hasRole("ADMIN")
80         .anyRequest().authenticated());
81
82     http.oauth2ResourceServer(oauth2 -> oauth2.jwt(
83         jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())));
84
85     http.csrf(csrf -> csrf.disable());
86     http.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
87
88     return http.build();
89 }
90

```

apiFilterChain (HttpSecurity http): El método establece el filtro de seguridad secundario (@Order(2)) que protege las rutas operativas de la API (/api/employees), aplicando reglas de Autorización Basada en Roles (RBAC) según el tipo de petición HTTP (GET, POST, DELETE), y configura a Spring Security como un Resource Server de OAuth2 para que exija y valide un token JWT en lugar de credenciales básicas.

```

91 @Bean
92 public JwtEncoder jwtEncoder() {
93     JWK jwk = new RSAKey.Builder(publicKey).privateKey(privateKey).build();
94     JWKSource<SecurityContext> jwks = new ImmutableJWKSet<>(new JWKSet(jwk));
95     return new NimbusJwtEncoder(jwks);
96 }
97
98 public JwtDecoder jwtDecoder() {
99     return NimbusJwtDecoder.withPublicKey(publicKey).build();
100 }
101

```

jwtEncoder(): El método construye y expone el componente responsable de fabricar y firmar criptográficamente los nuevos tokens JWT una vez que el usuario se autentica con éxito, empaquetando tanto la clave pública como la clave privada RSA de la aplicación en un almacén de claves utilizado por la librería interna Nimbus.

jwtDecoder(): El método configura el decodificador que intercepta los tokens entrantes en las peticiones de la API, utilizando únicamente la clave pública RSA inyectada para verificar matemáticamente la firma del JWT y asegurar que no haya sido manipulado por terceros.

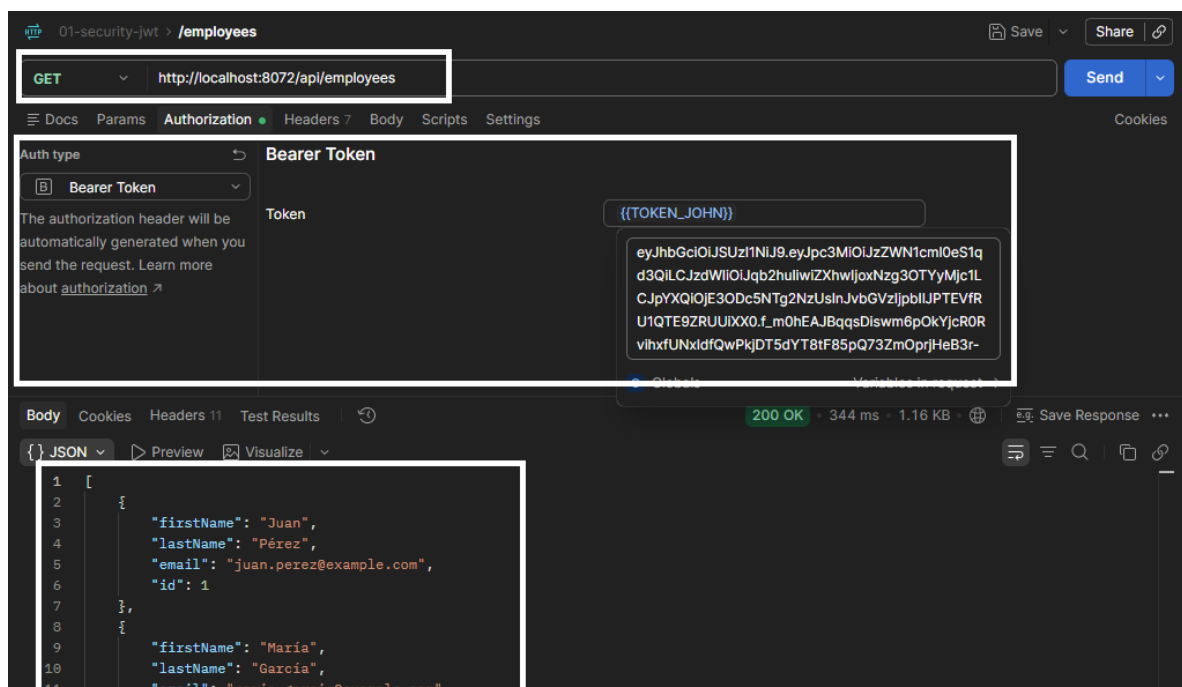
```
102 private JwtAuthenticationConverter jwtAuthenticationConverter() {
103
104     JwtGrantedAuthoritiesConverter authoritiesConverter = new JwtGrantedAuthoritiesConverter();
105     authoritiesConverter.setAuthoritiesClaimName("roles");
106     authoritiesConverter.setAuthorityPrefix("");
107
108     JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
109     converter.setJwtGrantedAuthoritiesConverter(authoritiesConverter);
110
111     return converter;
112 }
```

jwtAuthenticationConverter(): El método personaliza cómo Spring Security extrae la información de autorización del JWT decodificado, instruyendo al sistema para que busque los permisos del usuario dentro de un atributo (claim) llamado específicamente roles y evite agregarle el prefijo automático SCOPE_ que viene por defecto, permitiendo que coincida exactamente con los roles definidos en la base de datos.

```
50 @PostMapping("/login")
51 public TokenResponse login(Authentication authentication) {
52     Instant ahora = Instant.now();
53     // los roles que salieron de la tabla "roles": ROLE_EMPLOYEE, ROLE_MANAGER...
54     //
55     // El filtro startsWith("ROLE_") NO es decorativo. Spring Security 7 agrega por su
56     // cuenta autoridades que describen COMO te autenticaste (FACTOR_PASSWORD, y otras
57     // FACTOR * si usas multifactor). Son útiles dentro del servidor, pero no tienen
58     // nada que hacer dentro de un token que viaja al cliente. Sin este filtro, el
59     // token saldría con "roles":["ROLE_EMPLOYEE","FACTOR_PASSWORD"].
60     List<String> roles = authentication.getAuthorities().stream()
61         .map(GrantedAuthority::getAuthority)
62         .filter(authority -> authority.startsWith("ROLE_"))
63         .collect(Collectors.toList());
64
65     // El PAYLOAD del token. Toda esta viaja en claro dentro del token:
66     // ya FIRMADO, no cifrado. Nunca pongas aquí datos secretos.
67     JwtClaimsSet claims = JwtClaimsSet.builder()
68         .issuer("security-jwt") // quien lo emite
69         .issuedAt(ahora) // cuando (claim "iat")
70         .expiresAt(ahora.plusSeconds(ttlSeconds)) // hasta cuando (claim "exp")
71         .subject(authentication.getName()) // de quien es (claim "sub")
72         .claim("roles", roles) // que puede hacer
73         .build();
74
75     // firmar con RS256 = RSA + SHA-256, usando la llave privada
76     JwsHeader header = JwsHeader.with(org.springframework.security.oauth2.jose.jws.SignatureAlgorithm.RS256).build();
77     String token = jwtEncoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
78     return new TokenResponse(token, "Bearer", ttlSeconds, authentication.getName());
79 }
```

TokenResponse es un controlador de Spring Boot para el inicio de sesión que recibe los datos de un usuario previamente autenticado, extrae sus permisos que comienzan con "ROLE_" y genera un token JWT seguro. Para lograrlo, construye el contenido del token (claims) asignando el emisor, la fecha de emisión, la fecha de expiración, el identificador del usuario y los roles filtrados, para luego firmarlo criptográficamente usando el algoritmo asimétrico RS256. Finalmente, ensambla y codifica el token, devolviendo al cliente una respuesta que contiene el JWT listo para usarse, el prefijo de autorización "Bearer", su tiempo de vida útil en segundos y el nombre del usuario.

La consecuencia (Recuadro inferior derecho): El rechazo inmediato por parte del servidor con un estado 401 Unauthorized, confirmando que el filtro de Spring Security está haciendo su trabajo al proteger la API.



Envío del Token (Recuadro medio): En la pestaña de Autorización, el tipo ha cambiado a Bearer Token. Se está inyectando el token JWT (en este caso, almacenado en una variable de Postman llamada {{TOKEN_JOHN}} que contiene la cadena generada en el paso de login). Esto hace que Postman envíe automáticamente la cabecera HTTP Authorization: Bearer eyJhb....

Acceso concedido (Recuadro inferior): El servidor intercepta el token mediante el decodificador (jwtDecoder), valida su firma matemática y verifica los permisos. Al ser un pase válido, permite el acceso, responde con un estado 200 OK y devuelve en el cuerpo la lista de empleados en formato JSON. En esta transacción, las contraseñas reales nunca viajaron por la red.

Autenticación OAUTH2

¿Qué es?

Es un marco de autorización (más que un simple método de autenticación) que permite a una aplicación de terceros obtener acceso limitado a los recursos de un usuario sin necesidad de exponer su contraseña.

Problema que resuelve

Resuelve el problema de la delegación de permisos y el inicio de sesión federado. Permite que los usuarios inicien sesión en tu aplicación utilizando proveedores externos de confianza (como Google, GitHub, Keycloak o Auth0) sin que tu sistema tenga que gestionar ni almacenar contraseñas directamente.

Qué pasa si no se usa

Las aplicaciones se verían obligadas a implementar y mantener sus propios sistemas de registro de usuarios, recuperación de contraseñas y encriptación de credenciales desde cero. Además, sería imposible permitir de forma segura el acceso delegado a plataformas de terceros o integrar accesos rápidos con cuentas corporativas o de redes sociales (Single Sign-On).

```
21 @Configuration
22 public class SecurityConfig {
23
24     @Bean
25     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
26         http.authorizeHttpRequests(configurer -> configurer
27             .requestMatchers(HttpMethod.GET, "/api/employees").hasRole("EMPLOYEE")
28             .requestMatchers(HttpMethod.GET, "/api/employees/**").hasRole("EMPLOYEE")
29             .requestMatchers(HttpMethod.POST, "/api/employees").hasRole("MANAGER")
30             .requestMatchers(HttpMethod.PUT, "/api/employees").hasRole("MANAGER")
31             .requestMatchers(HttpMethod.PATCH, "/api/employees/**").hasRole("MANAGER")
32             .requestMatchers(HttpMethod.DELETE, "/api/employees/**").hasRole("ADMIN")
33             .anyRequest().authenticated());
34         http.oauth2ResourceServer(oauth2 -> oauth2.jwt(
35             jwt -> jwt.jwtAuthenticationConverter(keycloakConverter())));
36         http.csrf(csrf -> csrf.disable());
37         http.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
38         return http.build();
39     }
}
```

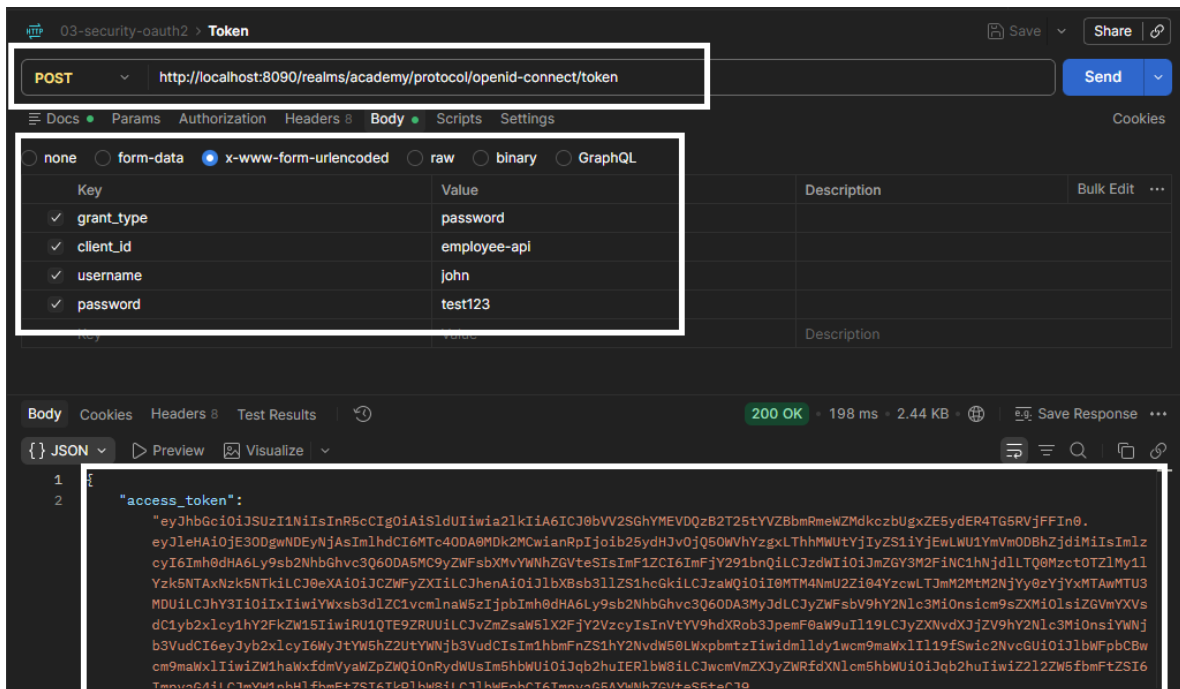
filterChain(HttpSecurity http): Este método configura las reglas principales de seguridad de la aplicación, definiendo un control de acceso basado en roles para proteger los endpoints de /api/employees según la acción solicitada (lectura exclusiva para EMPLOYEE, escritura/modificación para MANAGER, y borrado para ADMIN). Además, establece que la API operará de forma sin estado (stateless) y sin protección CSRF, actuando como un Servidor de Recursos OAuth2 que valida las

peticiones mediante tokens JWT procesados por un convertidor personalizado adaptado para Keycloak.

```
41 private Converter<Jwt, AbstractAuthenticationToken> keycloakConverter() {  
42     JwtAuthenticationConverter converter = new JwtAuthenticationConverter();  
43     converter.setJwtGrantedAuthoritiesConverter(SecurityConfig::extraerRoles);  
44     return converter;  
45 }  
46  
47 @SuppressWarnings("unchecked")  
48 private static Collection<GrantedAuthority> extraerRoles(Jwt jwt) {  
49     Map<String, Object> realmAccess = jwt.getClaim("realm_access");  
50     if (realmAccess == null || realmAccess.get("roles") == null) {  
51         return List.of();  
52     }  
53     List<String> roles = (List<String>) realmAccess.get("roles");  
54     return roles.stream()  
55         .map(role -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + role))  
56         .toList();  
57 }  
58 }
```

keycloakConverter(): Este método configura y devuelve el componente responsable de transformar el token JWT entrante en un objeto de autenticación que Spring Security pueda entender. Su propósito principal es sobrescribir el comportamiento por defecto del framework, enlazando la conversión de autoridades a un método personalizado (extraerRoles) para asegurar que el sistema lea los permisos desde la ubicación no estándar que utiliza Keycloak.

extraerRoles(Jwt jwt): Este método analiza el cuerpo del token generado por Keycloak para localizar y extraer los permisos del usuario, los cuales vienen anidados específicamente dentro de un objeto JSON llamado realm_access bajo la clave roles. Si encuentra esta lista, mapea cada cadena de texto agregándole el prefijo ROLE_ (convirtiendo, por ejemplo, ADMIN en ROLE_ADMIN), requisito indispensable para que las validaciones .hasRole() funcionen correctamente en el filtro de seguridad.



La imagen muestra una solicitud en Postman dirigida a un Servidor de Autorización (en este caso Keycloak) para obtener un token de acceso mediante el flujo de OAuth2.

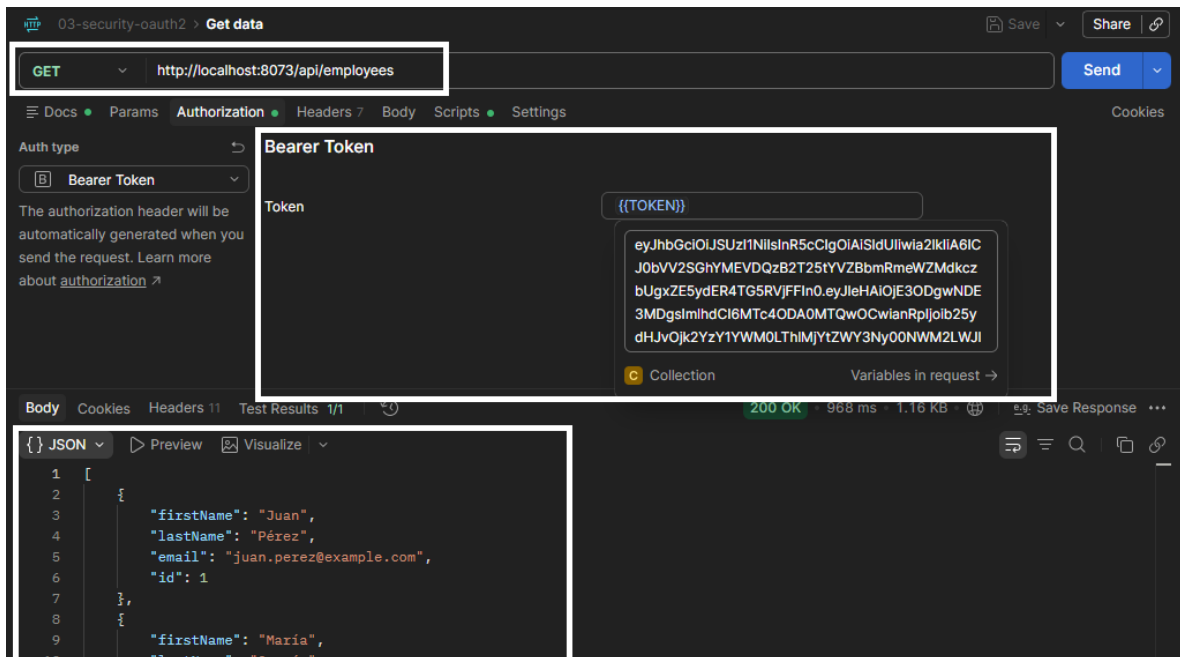
Aquí tienes el desglose de los elementos resaltados:

Petición al Servidor de Autorización (Recuadro superior): Se está realizando una solicitud POST al endpoint de generación de tokens de Keycloak: `http://localhost:8090/realms/academy/protocol/openid-connect/token`. Observa que la petición ya no va dirigida a tu API (puerto 8072 en ejemplos anteriores), sino al gestor central de identidades (puerto 8090).

Credenciales y Configuración (Recuadro medio): A diferencia de HTTP Basic donde los datos van en la pestaña Authorization, en OAuth2 los datos se envían en el cuerpo de la petición (Body) usando el formato `x-www-form-urlencoded`. Los parámetros clave son:

- **grant_type:** `password`: Indica que se usará el flujo donde el usuario entrega sus credenciales directamente (Resource Owner Password Credentials).
- **client_id:** `employee-api`: Identifica a qué aplicación se le está concediendo el acceso.
- **username y password:** Las credenciales reales del usuario (`john / test123`).

Respuesta y Token JWT (Recuadro inferior): Keycloak valida las credenciales y responde con un estado 200 OK. En el cuerpo del JSON entrega un `access_token` (un JWT muy largo). Este es el "pase de valet" que el cliente ahora copiará y enviará a tu API (el Resource Server) como un token tipo Bearer para poder consumir las rutas protegidas.



La imagen ilustra el paso final del flujo OAuth2: el consumo exitoso de una API protegida (el Resource Server) utilizando el token obtenido previamente del servidor de autorización (Keycloak).

Petición al Servidor de Recursos (Recuadro superior): El cliente realiza una solicitud GET a la ruta protegida de tu API: `http://localhost:8073/api/employees`. Nota que aquí volvemos a apuntar a tu aplicación (puerto 8073) y no a Keycloak.

Inyección del Token (Recuadro medio): En la pestaña de Authorization, se selecciona el tipo Bearer Token. Se está utilizando una variable de Postman llamada `{{TOKEN}}` que contiene el largo JWT generado en el paso anterior. Postman se encarga de inyectar esto en la cabecera HTTP de la petición.

Acceso Concedido (Recuadro inferior): La API de Spring Boot (configurada como OAuth2 Resource Server) intercepta la petición, extrae el token, verifica su firma digital usando la clave pública de Keycloak y valida que el usuario tenga el rol requerido. Al ser válido, responde con un 200 OK y devuelve la lista de empleados en formato JSON. Todo esto ocurre sin que tu API haya visto jamás la contraseña del usuario.